

CS1CA Computer Systems Architecture

Memory hierarchy Design (1)

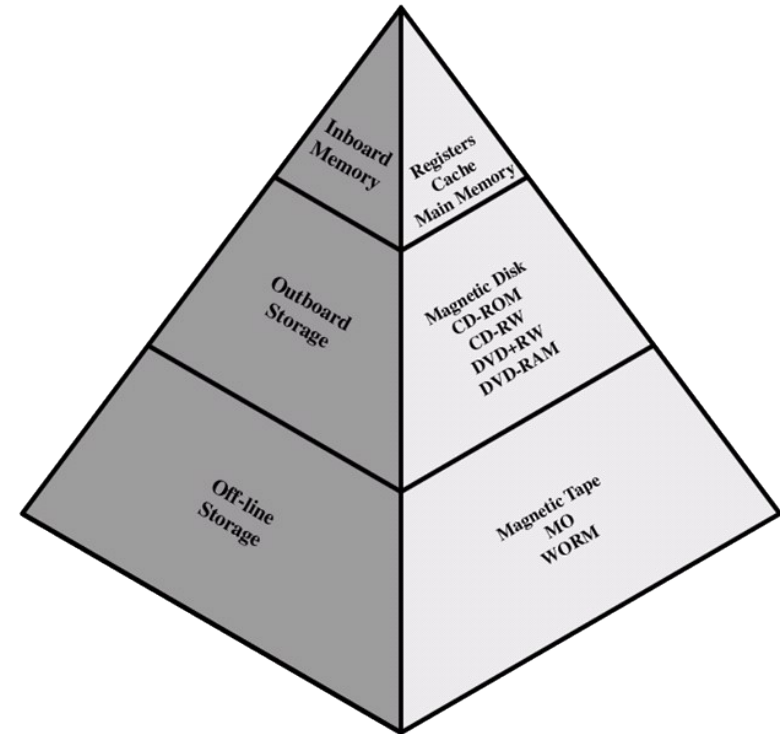
Week 10 lecture 1 of autumn semester 2025

Lectured by

Dr. Hong Wei

h.wei@reading.ac.uk

Room 133 in Polly Vacher



This topic will be delivered in weeks 10-11. In these two weeks, there are two lectures each week.

Outline of the topic

- General introduction to memory hierarchy
- Cache and cache performance measure
 - Techniques for reducing cache miss penalty
 - Techniques for reducing cache misses
 - Techniques for reducing hit time
- Main memory: different organisations
- Virtual memory
- Self-learning: Memory hierarchy design in the Alpha 21264

In this lecture

- The following contents will be delivered
 - General introduction to memory hierarchy
 - Principle of locality
 - Four issues in memory hierarchy of data transfer between cache and main memory
- The learning outcomes are listed below
 - To be able to articulate the principle of locality
 - To be able to explain the four issues in memory hierarchy

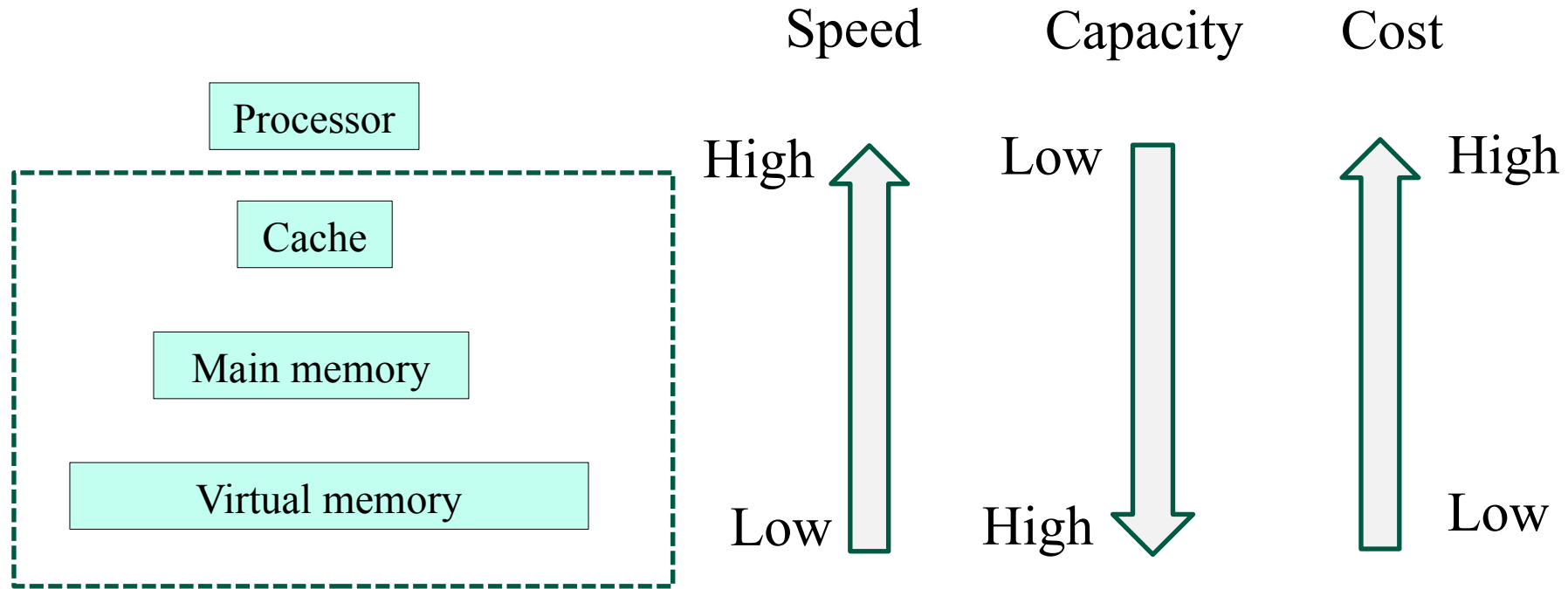
Why do we need memory hierarchy?

- Amdahl's law requires us to consider all components in improving performance.
- Requirements for unlimited amounts of fast memory;
- To improve performance while keeping memory system affordable.
- The idea came from the *principle of locality*.

The principle of locality

- Memory hierarchy operates on *the principle of locality*.
- The locality states that most programs do not access all code or data with equal probability;
- Temporal locality: frequency to reference an item. One item is referenced, it will have higher possibility to be referenced again soon.
- Spatial locality: area to reference items. One item is referenced, its neighbour items will have higher possibility to be referenced soon.
- A guideline for hardware design: the smaller, the faster.

A basic structure of memory hierarchy



Three levels in a memory hierarchy beyond processor:
cache → main memory → virtual memory (VM)

Caches

- The first level of the memory hierarchy encountered once the address leaves CPU is cache.
- **Block**: the minimum unit of information that can or can not be present in the cache.
 - **Cache hit**: block available in cache
 - **Cache miss**: block not present in cache
- Information in RAM is divided into **blocks**, which are copied from RAM to cache, and processor checks cache first with every memory reference.

LW R1, 200(R0)
Access a cache first.

Four issues in the memory hierarchy

- Where can a block be placed in the upper level?
(**block placement**)
- How is a block found if it is in the upper level?
(**block identification**)
- Which block should be replaced on a miss?
(**block replacement**)
- What happens on a write? (**write strategy**)

Block placement: three categories of cache organisation

Direct mapped:

$(\text{Block address}) \bmod (\text{no. of blocks in cache})$

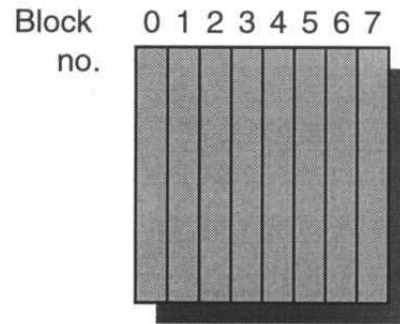
Set associative:

$(\text{Block address}) \bmod (\text{no. of sets in cache})$

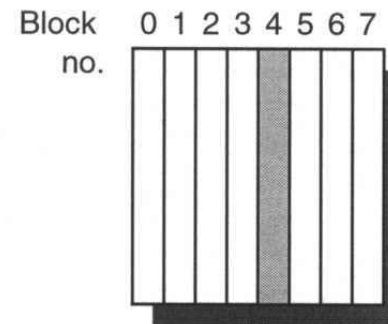
8-cache
blocks

Cache

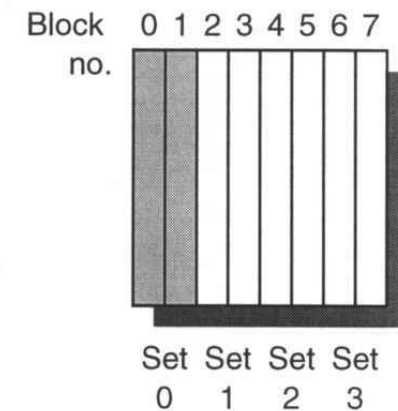
(1) Fully associative:
block 12 can go
anywhere



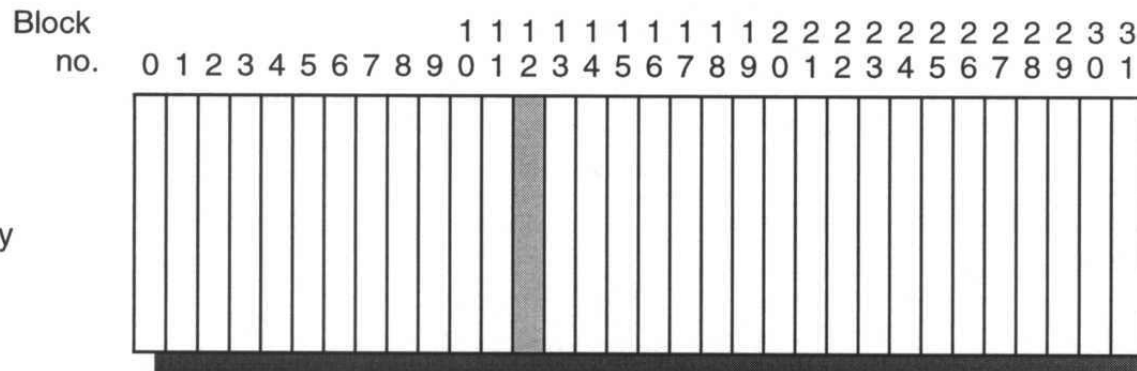
(2) Direct mapped:
block 12 can go
only into block 4
($12 \bmod 8$)



(3) Set associative:
block 12 can go
anywhere in set 0
($12 \bmod 4$)



Block frame address

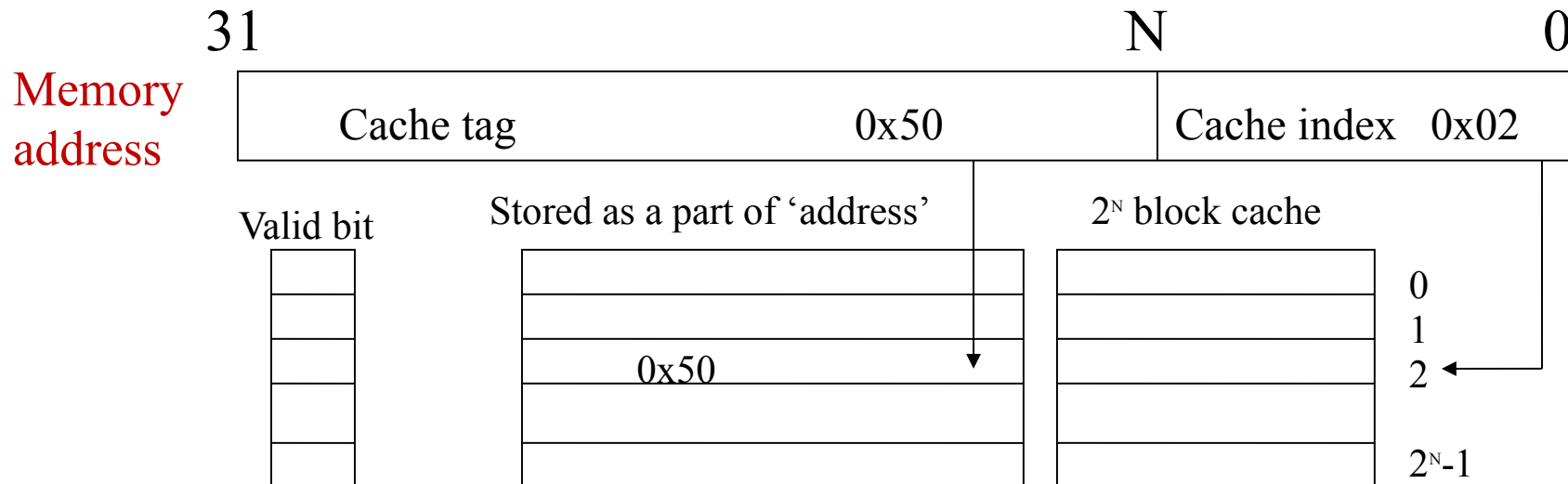


Memory

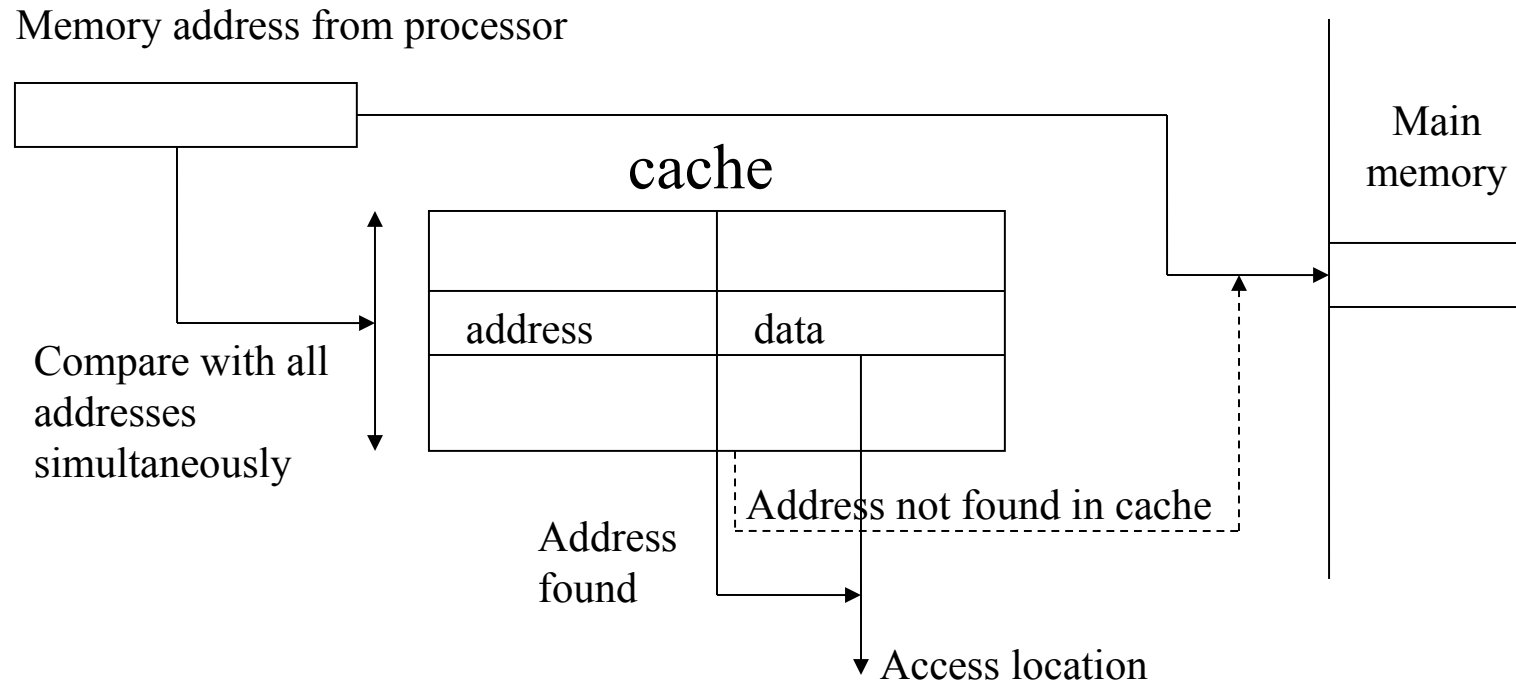
Block 12 is
copied from
RAM to cache.

Two terms: cache index and cache tag

- Assume a 32-bit memory address, which is split into two parts, cache index and cache tag.
- For a 2^N block direct mapped cache,
 - **Cache index**: the lower N bits of the memory address
 - **Cache tag**: the upper $(32-N)$ bits of the memory address

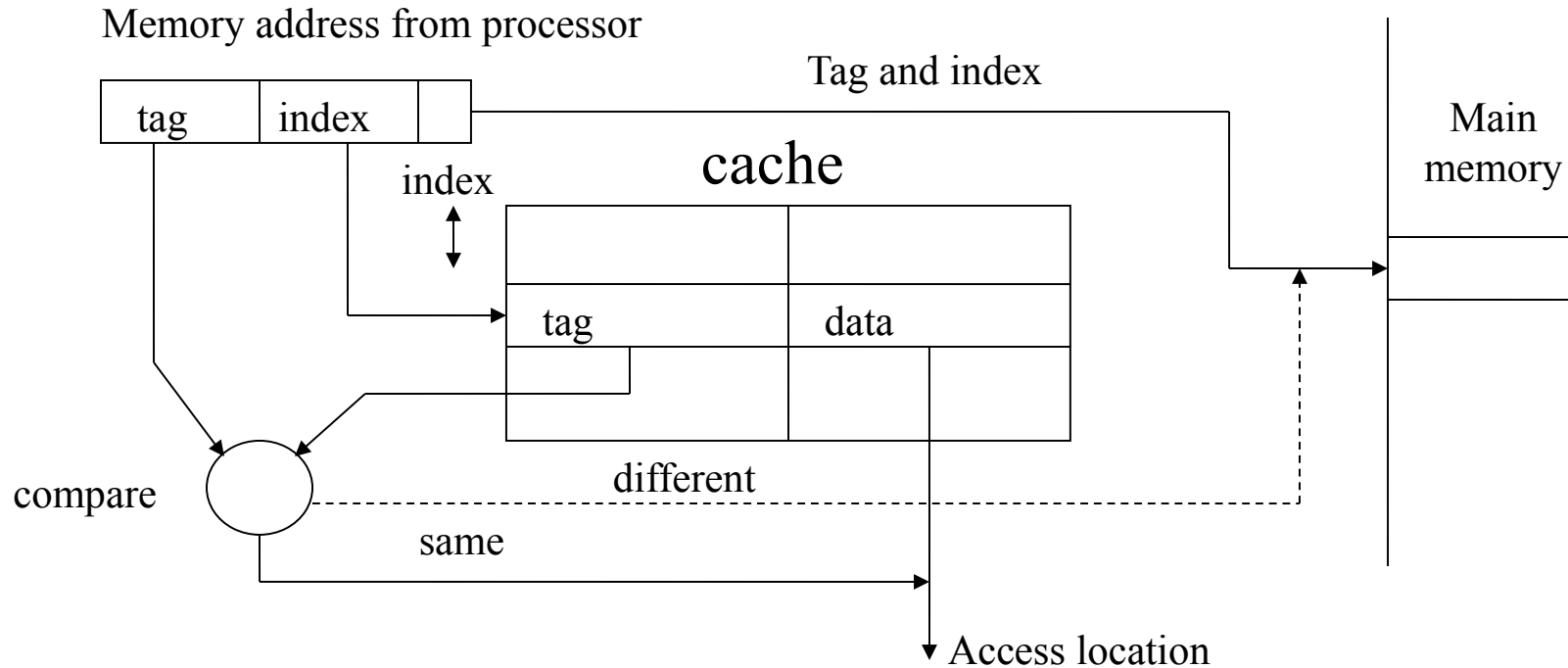


Block identification: fully associative



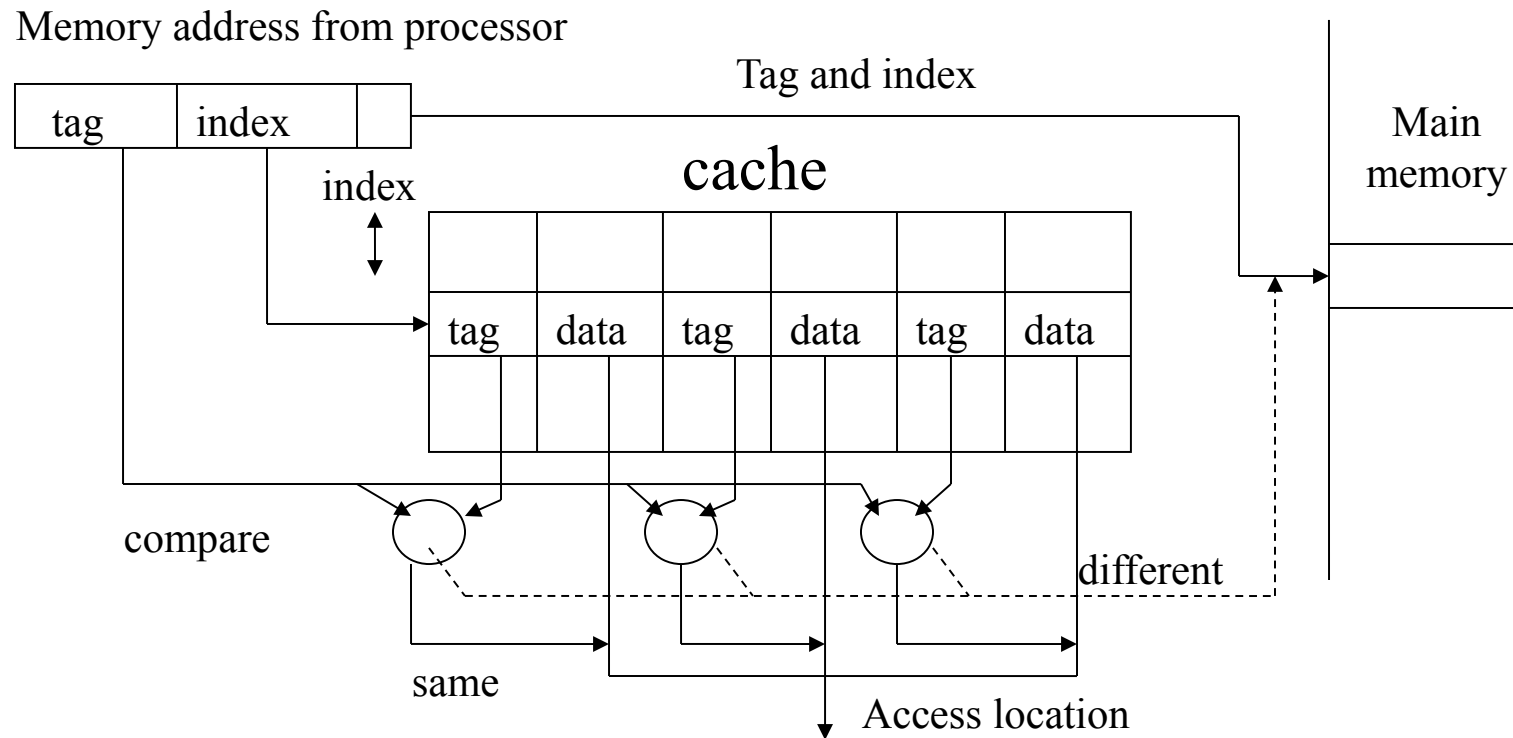
- A block can be placed anywhere in the cache;
- Constructed cache as content addressable memory i.e. memory address identifies block in cache.

Block identification: direct mapping



- Block position is found by an index:
(block address) MOD (no. of blocks in cache);
- Cache entries are tagged with upper part of memory address;
- Block can only be put at the index location.

Block identification: set associative



- Block position found via a set index:
(block address) MOD (no. of sets in cache)
- In the above example three blocks in the set *i.e.* 3-way set associative.

Block placement and identification: summary

- Suppose cache has m blocks;
- A range of caches from direct mapped to fully associative forms a continuum i.e. direct mapped is one-way associative (m sets), and fully associative is m -way set associative (one set);
- A set-associative or direct-mapped cache has three portions in an address.

Block address		Block offset
Tag	Index	

- **Tag** to check all blocks in the set, **Index** to find set, and **Block offset** to address the desired data within a block;
- Cache entries also have a valid bit associated with the tag;
- Most caches are direct mapped and 2- or 4-way associative in practice.

Block replacement – cache miss

- When a cache miss occurs, the cache controller must select a block to be discarded.
- For direct mapping there is no choice as only one block is checked for a hit.
- For set associative there are two strategies
 - Randomly select a block – easiest to build in hardware;
 - Least Recently Used (LRU) block is chosen for replacement. This may be implemented by associating a counter with a block. The counter is incremented at regular intervals when the cache is referenced. The counter is reset to zero if the block is accessed. The counter with the highest value indicates the LRU block.

Reads and writes

- **Reads**

- Block can be read at the same time that the tag is read and compared;
- **Cache hit:** block is passed to CPU;
- **Cache miss:** ignore value read.

- **Writes (note: it is different from Reads.)**

- Cannot modify a block until tag is checked to see if address is a hit; therefore cannot write in parallel with tag checking;
- Takes longer time than read does;
- May need to write to a specific location within a block.

Two writing mechanisms

- *Write-through*: information is written to both blocks in cache and in main memory;
- *Write-back*: information is written to block in cache. Modified cache block is written to main memory only when it is replaced.
 - ❖ To reduce frequency of write-backs on replacement, a *dirty* bit is used as a status. The status indicates a block is modified (dirty) or not (clean) in the cache. If the block is dirty, it should be written back to main memory on a miss.

Advantages of write-back mechanism

- Writes occur at the speed of the cache memory. Therefore they are fast;
- Multiple writes to a block in cache only require one write to the main memory;
- Uses less memory bandwidth. This is attractive for multiprocessors.

Advantage of write-through mechanism

- Easier to implement than write back;
- Main memory has current copy of data. This is an advantage for operating systems and I/O;
- **Write stall**: Processor waits for writes to complete during a write through.
 - A common optimisation to reduce write stalls is **write buffer**, which allows the processor to continue as soon as the data is written to the buffer.

End of this lecture

Questions and exercises

- What is the concept of memory hierarchy design and what is the principle of locality?
- What are the four issues in memory hierarchy? (refer to data transfer between cache and main memory)
- How to address the four issues in practice?